CSA17- ARTIFICIAL INTELLIGENCE

Lab Experiments

1.Write the python program to solve 8-Puzzle problem

Aim

To solve the **8-Puzzle problem using Breadth-First Search (BFS)**.

Algorithm

1. Initialize the start state and goal state.
2. Store the start state in a queue.
3. Find the blank space (0) and generate possible moves.
4. Add unvisited states to the queue.
5. Repeat until the goal state is reached.
6. Display the solution path and number of moves.

Code:

```
from collections import deque

# Initial puzzle

start = (1, 2, 3,

        4, 0, 6,

        7, 5, 8)

# Goal puzzle

goal = (1, 2, 3,

        4, 5, 6,

        7, 8, 0)
```

```
# Possible positions for blank (0)
moves = {
        0: [1, 3],
        1: [0, 2, 4],
        2: [1, 5],
        3: [0, 4, 6],
        4: [1, 3, 5, 7],
        5: [2, 4, 8],
        6: [3, 7],
        7: [4, 6, 8],
        8: [5, 7]
}
def solve(start):
        queue = deque([(start, [start])])
        visited = {start}
        while queue:
          state, path = queue.popleft()
        if state == goal:
        return path
        blank = state.index(0)
        for pos in moves[blank]:
              new_state = list(state)
        # Swap blank with next position
```

```
            new_state[blank], new_state[pos] = \
                    new_state[pos], new_state[blank]
              new_state = tuple(new_state)
            if new_state not in visited:
                    visited.add(new_state)
                    queue.append((new_state, path + [new_state]))
            return None
# Solve puzzle
solution = solve(start)
# Display solution
print("8-Puzzle Solution:\n")
for state in solution:
            print(state[0], state[1], state[2])
            print(state[3], state[4], state[5])
            print(state[6], state[7], state[8])
            print()
print("Minimum moves:", len(solution) - 1)
```

OUTPUT:

```
>>>
8-Puzzle Solution:

1 2 3
4 0 6
7 5 8

1 2 3
4 5 6
7 0 8

1 2 3
4 5 6
7 8 0

Minimum moves: 2
>>>
```

Result

Thus, the **8-Puzzle problem was successfully solved using BFS**, and the minimum sequence of moves was obtained.

2. Write the python program to solve 8-Queen problem

Aim

To solve the **8-Queens problem using the Backtracking technique**.

Algorithm

1. Start from the first row.
2. Place a queen in a safe column.
3. Check whether the queen conflicts with previous queens.
4. If safe, move to the next row.
5. If no safe position exists, backtrack.
6. Continue until all 8 queens are placed.

Code:

```
N = 8

board = [-1] * N

def is_safe(row, col):

        for i in range(row):

        if board[i] == col:

        return False

        if abs(board[i] - col) == abs(i - row):

        return False

        return True

def solve(row):

        if row == N:

        return True

        for col in range(N):
```

```
        if is_safe(row, col):
        board[row] = col
        if solve(row + 1):
                return True
        board[row] = -1
        return False
# Solve the problem
if solve(0):
    print("8-Queens Solution:\n")
        for row in range(N):
        for col in range(N):
        if board[row] == col:
                print("Q", end=" ")
        else:
                print(".", end=" ")
        print()
else:
        print("No solution exists")
```

OUTPUT:

```
>>> ==================================== RE:
>>>
8-Queens Solution:

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

Result

Thus, the **8-Queens problem was successfully solved using Backtracking**, obtaining a valid arrangement of 8 queens.

3. Write the python program for Water Jug Problem

Aim

To solve the **Water Jug Problem using Breadth-First Search (BFS)**.

Algorithm

1. Start with both jugs empty.
2. Generate possible operations: fill, empty, and transfer water.
3. Store each state in a queue.
4. Avoid already visited states.
5. Continue until the required amount of water is obtained.

Code:

```
from collections import deque

# Jug capacities

A = 4

B = 3

# Initial state

start = (0, 0)

# Goal

goal = 2

def solve():

        queue = deque([(start, [])])

        visited = set()

        while queue:

         (a, b), path = queue.popleft()

        if (a, b) in visited:
```

```
        continue
        visited.add((a, b))
        path = path + [(a, b)]
        # Goal reached
        if a == goal or b == goal:
        return path
        states = [
        (A, b),          # Fill A
        (a, B),          # Fill B
        (0, b),          # Empty A
        (a, 0),          # Empty B
        (a - min(a, B-b), b + min(a, B-b)),  # A -> B
        (a + min(b, A-a), b - min(b, A-a))   # B -> A
        ]
        for state in states:
        if state not in visited:
            queue.append((state, path))
        return None
solution = solve()
print("Water Jug Solution:\n")
for state in solution:
        print(state)
print("\nGoal: 2 litres")
```

OUTPUT:

```
>>>
Water Jug Solution:

(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)

Goal: 2 litres
>>>
```

Result

Thus, the **Water Jug Problem was successfully solved using BFS**, obtaining **2 litres of water**.

4. Write the python program for Cript-Arithmetic problem.

Aim

To solve a **Crypt-Arithmetic problem using Python** by assigning unique digits to letters.

Algorithm

1. Assign different digits (0–9) to each letter.
2. Ensure the first letters are not zero.
3. Form the numerical values of the words.
4. Check whether SEND + MORE = MONEY.
5. Print the valid assignment.

Code:

```
from itertools import permutations

# Crypt-Arithmetic: SEND + MORE = MONEY

letters = 'SENDMORY'

for p in permutations(range(10), len(letters)):

        S, E, N, D, M, O, R, Y = p

        # S and M cannot be zero

        if S == 0 or M == 0:

        continue

        SEND = 1000*S + 100*E + 10*N + D

        MORE = 1000*M + 100*O + 10*R + E

        MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

        if SEND + MORE == MONEY:

        print("Solution found:")

        print("SEND =", SEND)
```

```
        print("MORE =", MORE)

        print("MONEY =", MONEY)

        break
```

OUTPUT:



Result

Thus, the Crypt-Arithmetic problem was successfully solved, obtaining: 9567085 = 10652.

5.Write the python program for Missionaries Cannibal problem

Aim

To solve the **Missionaries and Cannibals problem using BFS**.

Algorithm

1. Start with 3 missionaries and 3 cannibals on the left bank.
2. Generate possible boat movements.
3. Check that missionaries are never outnumbered by cannibals.
4. Store unvisited states in a queue.
5. Continue until everyone reaches the right bank.

Code:

```
from collections import deque

start = (3, 3, 0)

goal = (0, 0, 1)

moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

def safe(m, c):

    if m < 0 or c < 0 or m > 3 or c > 3:

    return False

    if m > 0 and m < c:

    return False

    rm = 3 - m

    rc = 3 - c

    if rm > 0 and rm < rc:

    return False

    return True
```

```
def solve():
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
    state, path = queue.popleft()
    m, c, boat = state
    if state == goal:
    return path
    for dm, dc in moves:
    if boat == 0:
        new_state = (m - dm, c - dc, 1)
    else:
        new_state = (m + dm, c + dc, 0)
    if safe(new_state[0], new_state[1]) and new_state not in visited:
        visited.add(new_state)
        queue.append((new_state, path + [new_state]))
    return None
solution = solve()
print("Missionaries and Cannibals Solution:\n")
for state in solution:
    print(state)
print("\n(M, C, Boat)")
print("Boat: 0 = Left, 1 = Right")
```

OUTPUT:

```
>>>
Missionaries and Cannibals Solution:

(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)

(M, C, Boat)
Boat: 0 = Left, 1 = Right
>>>
```

Result

Thus, the Missionaries and Cannibals problem was successfully solved using BFS.

6.Write the python program for Vacuum Cleaner problem

Aim

To solve the Vacuum Cleaner Problem using BFS.

Algorithm

1. Start with both rooms dirty.
2. Check the current room.
3. Clean the room if it is dirty.
4. Move the vacuum to the other room.
5. Repeat until both rooms are clean.

Code:

```
from collections import deque

start = ('A', 1, 1)

goal = ('A', 0, 0)

def solve():

        queue = deque([(start, [start])])

        visited = {start}

        while queue:

          state, path = queue.popleft()

          position, left, right = state

        if left == 0 and right == 0:

        return path

                if position == 'A' and left == 1:

        new_state = ('A', 0, right)

        elif position == 'B' and right == 1:
```

```
            new_state = ('B', left, 0)
            else:
            new_state = None
            if new_state and new_state not in visited:
            visited.add(new_state)
            queue.append((new_state, path + [new_state]))
            if position == 'A':
            new_state = ('B', left, right)
            else:
             new_state = ('A', left, right)
            if new_state not in visited:
              visited.add(new_state)
            queue.append((new_state, path + [new_state]))
            return None
solution = solve()
print("Vacuum Cleaner Solution:\n")
for state in solution:
            print(state)
```

OUTPUT:

```
>>>
Vacuum Cleaner Solution:

('A', 1, 1)
('A', 0, 1)
('B', 0, 1)
('B', 0, 0)
>>>
```

Result

Thus, the **Vacuum Cleaner Problem was successfully solved using BFS**, and both rooms were cleaned.

7.Write the python program to implement BFS.

Aim

To implement **Breadth-First Search (BFS)** for traversing a graph.

Algorithm

1. Start from the given node.
2. Add the node to the queue.
3. Remove a node from the front of the queue.
4. Visit its unvisited adjacent nodes and add them to the queue.
5. Repeat until the queue is empty.

Code:

```
from collections import deque

graph = {

        'A': ['B', 'C'],

        'B': ['D', 'E'],

        'C': ['F'],

        'D': [],

        'E': [],

        'F': []

}

start = 'A'

queue = deque([start])

visited = set()

print("BFS Traversal:")

while queue:
```

```
    node = queue.popleft()
    if node not in visited:
    print(node, end=" ")
    visited.add(node)
    for neighbour in graph[node]:
    if neighbour not in visited:
            queue.append(neighbour)
```

OUTPUT:



Result

Thus, BFS graph traversal was successfully implemented, and the nodes were visited level by level.

8.Write the python program to implement DFS.

Aim

To implement Depth-First Search (DFS) for traversing a graph.

Algorithm

1. Start from the given node.
2. Mark the node as visited.
3. Visit an unvisited adjacent node.
4. Continue recursively until no unvisited node remains.
5. Backtrack and visit remaining nodes.

Code:

```
graph = {
        'A': ['B', 'C'],
   'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': [],
        'F': []
}
visited = set()
def DFS(node):
        visited.add(node)
        print(node, end=" ")

        for neighbour in graph[node]:
```

```
        if neighbour not in visited:
            DFS(neighbour)
print("DFS Traversal:")
DFS('A')
```

OUTPUT:

```
>>>
DFS Traversal:
A B D E C F
>>>
```

Result

Thus, DFS graph traversal was successfully implemented, and the nodes were visited depth-wise.

9.Write the python to implement Travelling Salesman Problem

Aim

To implement the Travelling Salesman Problem (TSP) using the Brute Force method.

Algorithm

1. Store the distances between cities.
2. Generate all possible routes.
3. Calculate the cost of each route.
4. Compare all route costs.
5. Select the route with minimum cost.

Code:

```
from itertools import permutations

distance = [

        [0, 10, 15, 20],

        [10, 0, 35, 25],

        [15, 35, 0, 30],

        [20, 25, 30, 0]

]

n = len(distance)

cities = range(1, n)

min_cost = float('inf')

best_path = None

for path in permutations(cities):

        route = (0,) + path + (0)

        cost = 0
```

```
        for i in range(len(route) - 1):
            cost += distance[route[i]][route[i + 1]]
        if cost < min_cost:
            min_cost = cost
            best_path = route
print("Best Route:", end=" ")
for city in best_path:
    print(city, end=" ")
print("\nMinimum Cost:", min_cost)
```

OUTPUT:

```
>>> ================================ RESTART ================================
>>>
('Shortest Path:', (0, 1, 3, 2, 0))
('Minimum Distance:', 80)
>>>
```

Result

Thus, the Travelling Salesman Problem was successfully implemented, and the minimum-cost route was obtained.

10.Write the python program to implement A* algorithm

Aim

To implement the A* search algorithm to find the shortest path between two nodes.

Algorithm

1. Start with the initial node.
2. Calculate f(n) = g(n) + h(n).
3. Select the node with the lowest f(n).
4. Expand its neighboring nodes and calculate their costs.
5. Repeat until the goal node is reached.
6. Display the shortest path and its cost.

Code:

```
import heapq

graph = {

        'A': [('B', 1), ('C', 4)],

        'B': [('D', 2), ('E', 5)],

        'C': [('E', 1)],

        'D': [('G', 3)],

        'E': [('G', 2)],

        'G': []

}

h = {

        'A': 7,

        'B': 6,

        'C': 4,
```

```
        'D': 3,
        'E': 2,
        'G': 0
}
def astar(start, goal):
        queue = [(h[start], 0, start, [start])]
        visited = set()
        while queue:
        f, cost, node, path = heapq.heappop(queue)
        if node == goal:
        return path, cost
        if node in visited:
        continue
      visited.add(node)
        for neighbour, edge_cost in graph[node]:
        new_cost = cost + edge_cost
           f = new_cost + h[neighbour]
        heapq.heappush(
           queue,
           (f, new_cost, neighbour, path + [neighbour])
        )
        return None, None
start = 'A'
```

```
goal = 'G'

path, cost = astar(start, goal)

print("A* Algorithm")

print("Path:", " -> ".join(path))

print("Cost:", cost)
```

OUTPUT:

```
>>> ================================ RESTART ================================
>>>
('Path:', ['A', 'B', 'D', 'G'])
('Cost:', 5)
>>>
```

Result

Thus, the **A* algorithm was successfully implemented** and the shortest path from **A to G** was found.